

SEMANA 7

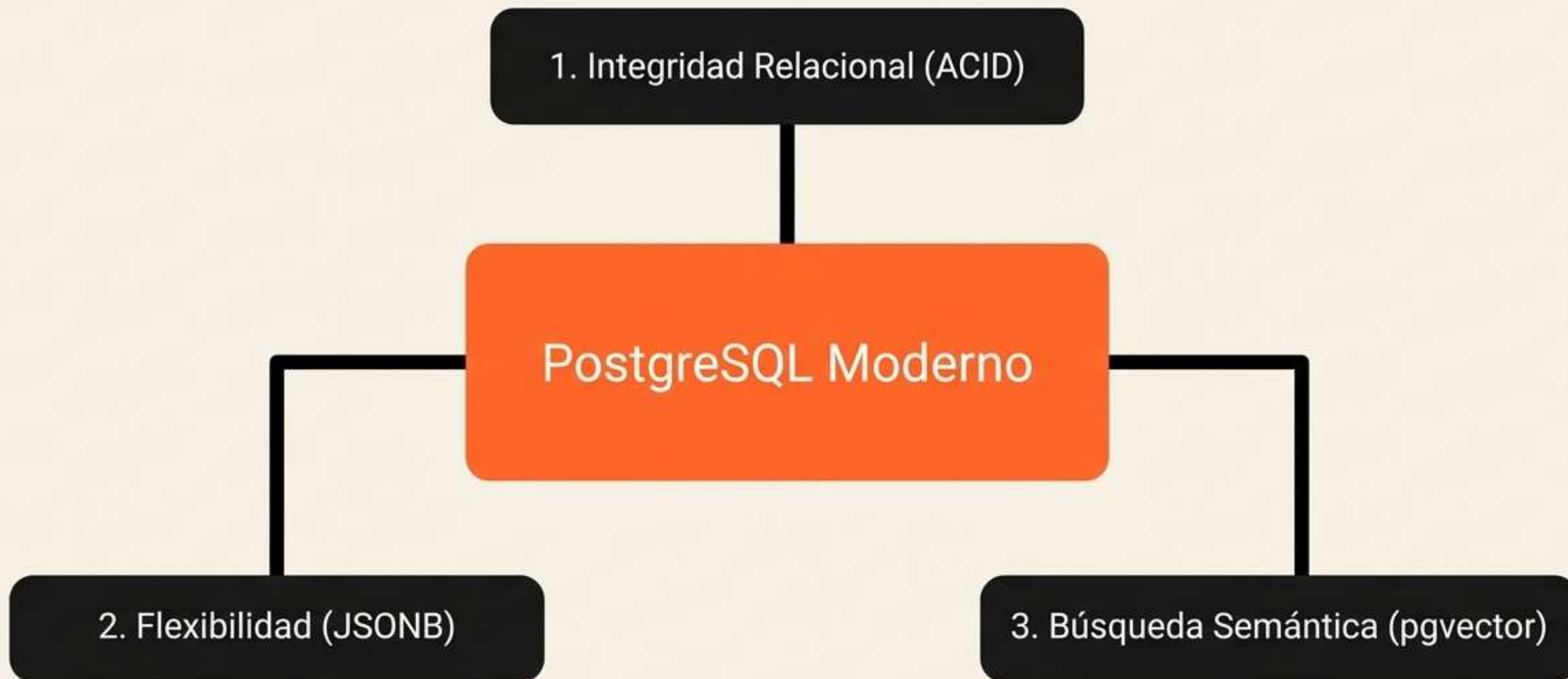
PostgreSQL Moderno

JSONB, Extensiones de IA
y Arquitecturas RAG



De motor relacional a plataforma de datos híbrida

Postgre ha evolucionado. Ya no necesitas múltiples bases de datos para manejar estructuras dinámicas y cargas de IA en producción. La unificación reduce costos operativos y elimina problemas de sincronización de datos.



JSON vs JSONB: La anatomía del almacenamiento

JSON (Texto Plano)	JSONB (Binario)
● Almacena el documento tal cual. ● Conserva espacios en blanco y orden original. ● Requiere re-parsing completo por cada consulta. Lento.	● Descompuesto binariamente. ● Nodos individuales indexables (GIN). ● Extremadamente rápido. Optimiza espacios redundantes.

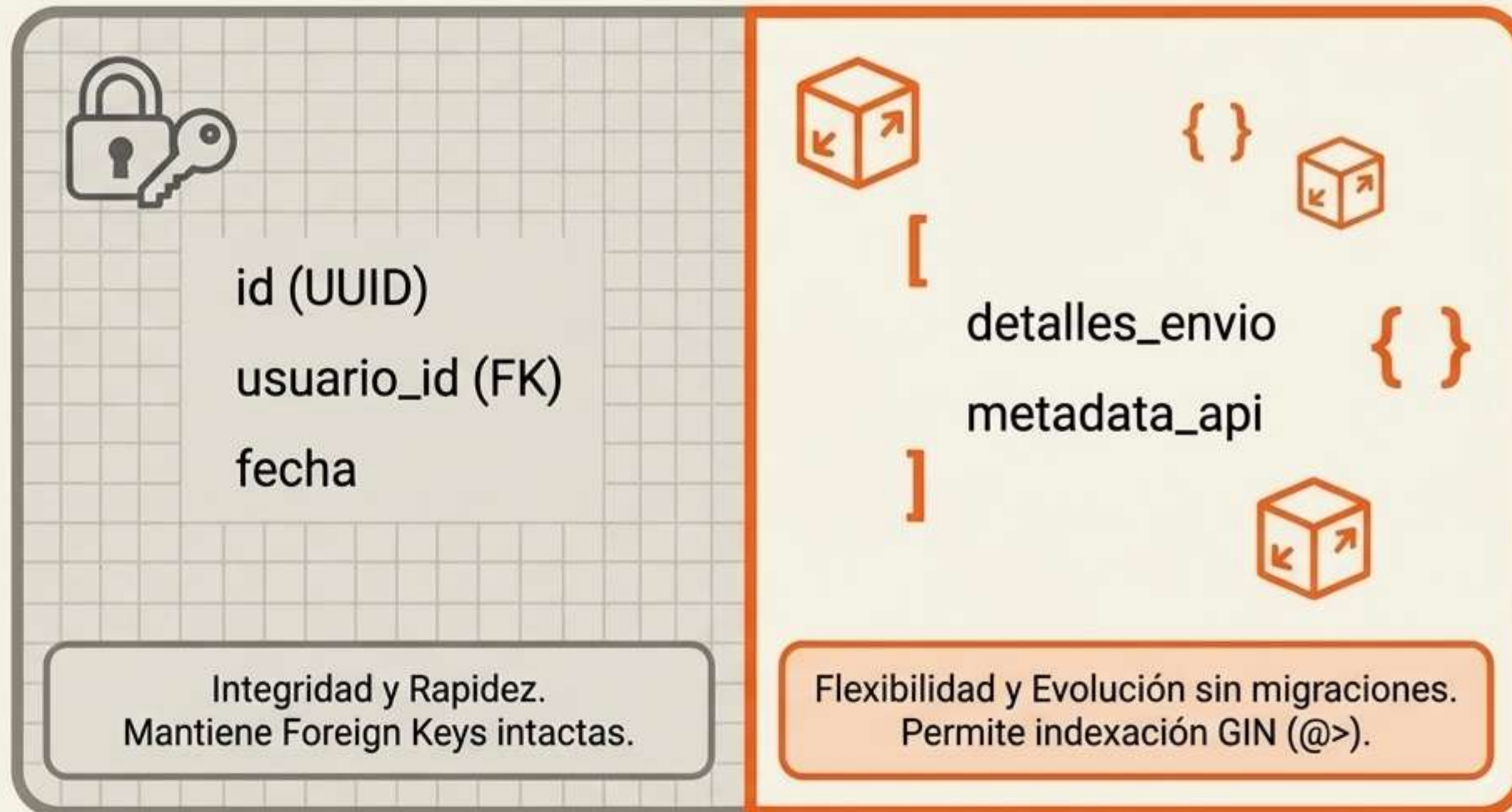


Coder Tips

Regla profesional 2026: Siempre usar JSONB. Excepción única: auditoría estricta de logs donde el formato exacto importa.

El patrón estándar de producción

Columnas relacionales para integridad + JSONB para atributos dinámicos.



Evita migraciones costosas por cambios en APIs externas (ej. Stripe, Shopify) almacenando las respuestas variables nativamente en JSONB.

Búsqueda Semántica Nativa con pgvector



Un embedding es un vector de N dimensiones que mapea el significado de un texto. pgvector permite almacenar e interrogar estos vectores nativamente en Postgres.

Para pensar:
¿pgvector vs VDB
dedicada?

Unificación ACID, sin
dual writes, y filtrado
nativo relacional
(WHERE). Es suficiente y
recomendado hasta
~10-50M de vectores.

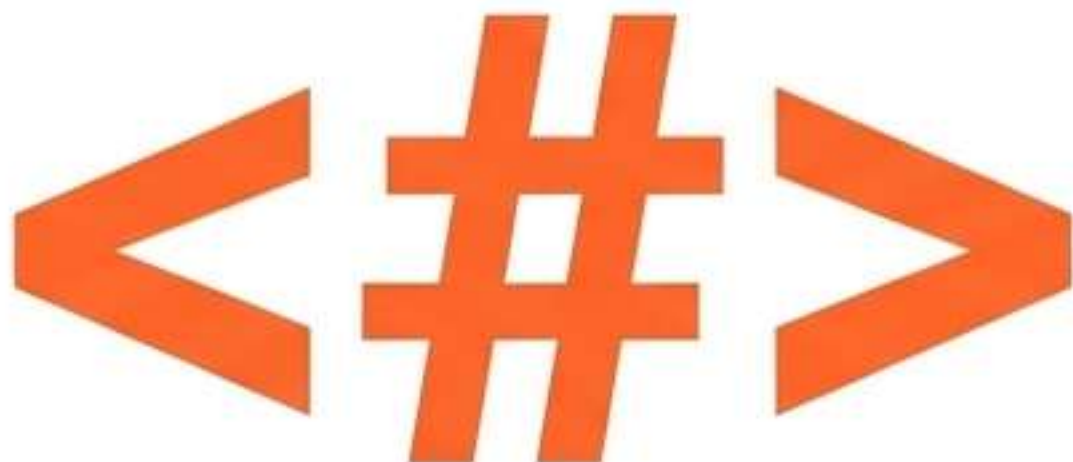
Operadores de Distancia Vectorial (Cheat Sheet)



(Coseno): El estándar absoluto para texto.
(La magnitud del vector no importa, solo el ángulo/dirección).



(Euclidiana L2): Ideal para distancia espacial/geométrica o si los vectores ya están normalizados.



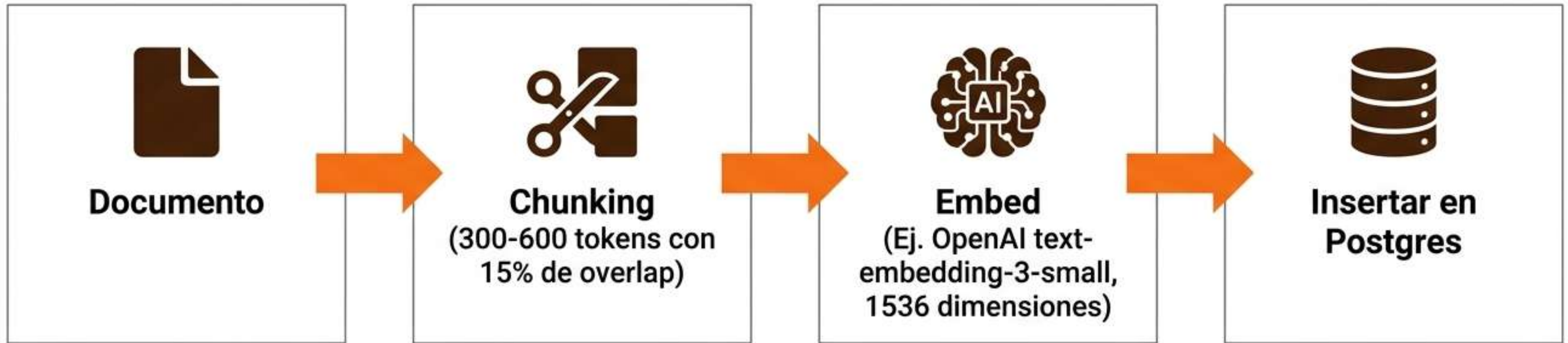
(Producto Punto Interno): El operador más rápido, pero solo debe usarse si tus embeddings ya están normalizados a longitud 1 (L2).

Arquitectura RAG Moderna: Búsqueda Híbrida

RRF compensa las debilidades individuales de ambos modelos para obtener la máxima precisión de recuperación.



Flujo de Trabajo RAG (De la Ingesta a la Recuperación)



Metadata

Guardar el nombre del modelo de embedding en una columna JSONB.



Índices

Usar HNSW (**iterative_scan = strict_order**) para máxima velocidad de recuperación en producción.

Filtros

Combinar lógica de usuario (relacional) con similitud vectorial en una sola query.

Material Ampliado: Checklist de Producción



Almacenamiento: Usar JSONB exclusivamente. Crear índices GIN en campos JSONB y FTS (tsvector generado).



Vectores: Configurar las dimensiones exactas del modelo (ej. VECTOR(1536)). Utilizar índices HNSW (versión pgvector 0.8+).



Seguridad (RBAC): Aplicar RLS (Row Level Security) para aislar datos en aplicaciones multi-tenant.



Mantenimiento: Ejecutar VACUUM ANALYZE post-carga masiva. Programar REINDEX trimestral para evitar bloat en índices GIN.



**¡Gracias por
estudiar
con nosotros!**